

Appendix 3.1 Model-View Controller (MVC) Pattern with Laravel

Model-View-Controller (MVC) is a common software architectural pattern that applies the principle of separation of concerns: you separate the data (i.e., the Model) from the output (i.e., the View), using the Controller as the agent.

MVC can be used for application design as well as Web design. The Models are normally represented by classes and database tables. For a Web application, the Views are the HTML pages, i.e., the final, dynamically generated output that the user actually sees. The Controllers react to user actions, such as the request of a single page or the submission of a form. Controllers implement the logic: validate data, insert into the database, show the results, etc.

By using MVC, projects will be easier to expand and easier to maintain. This is because you can add and change any of the three components without necessarily touching the others.

In this practical, you will be creating a simple web application using Laravel 12, a free, open-source framework for developing web applications according to the MVC architecture.

There are a variety of tools and frameworks available to you when building a web application. However, Laravel is the best choice for building modern, full-stack web applications.

A. Installing Laravel

Laravel uses **Composer** for managing dependencies.

1. Go to <https://getcomposer.org/download/>
2. Download and install **Composer-Setup.exe**
3. If Composer was successfully installed, you should see the following displayed when you type composer at the command prompt:

```
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>composer

          _____
         /  _   \  |
        / ___ \ | |
       / ___ \| |_|
      /_/   \_\___|
              |__|

Composer version 2.8.6 2025-02-25 13:03:50

Usage:
  command [options] [arguments]

Options:
  -h, --help                Display help for the given command. When no command is given display help for the l
command
  -q, --quiet               Do not output any message
  -V, --version             Display this application version
                          Force (or disable --no-ansi) ANSI output
  -n, --no-interaction     Do not ask any interactive question
  --profile                 Display timing and memory usage information
  --no-plugins              Whether to disable plugins.
  --no-scripts              Skips the execution of all scripts defined in composer.json file.
  -d, --working-dir=WORKING-DIR If specified, use the given directory as working directory.
  --no-cache                Prevent use of the cache
  -v|vv|vvv, --verbose     Increase the verbosity of messages: 1 for normal output, 2 for more verbose output
3 for debug
```

Note: Composer is a tool that includes all the dependencies and libraries. It allows a user to create a project with respect to the mentioned framework (for example, those used in a Laravel installation). Third-party libraries can be installed easily with the help of Composer.

All the dependencies are noted in the `composer.json` file, which is placed in the source folder.

B. Create Laravel Project

1. Go to your terminal window (command prompt)
2. Navigate to the **htdocs** folder
3. Type the following command:

```
composer create-project laravel/laravel ordersystem
```

Note: It will take a few minutes. Figure 2.1 shows the completed process.

```
- Installing symfony/polyfill-php83 (v1.31.0): Extracting archive
- Installing symfony/polyfill-mbstring (v1.31.0): Extracting archive
- Installing symfony/http-foundation (v7.2.3): Extracting archive
- Installing fruitcake/php-cors (v1.3.0): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing psr/http-client (1.0.3): Extracting archive
- Installing ralouphie/getallheaders (3.0.3): Extracting archive
- Installing psr/http-factory (1.1.0): Extracting archive
- Installing guzzlehttp/psr7 (2.7.0): Extracting archive
- Installing guzzlehttp/promises (2.0.4): Extracting archive
- Installing guzzlehttp/guzzle (7.9.2): Extracting archive
- Installing symfony/polyfill-php80 (v1.31.0): Extracting archive
- Installing guzzlehttp/uri-template (v1.0.4): Extracting archive
- Installing symfony/polyfill-intl-normalizer (v1.31.0): Extracting archive
- Installing symfony/polyfill-intl-grapheme (v1.31.0): Extracting archive
- Installing symfony/string (v7.2.0): Extracting archive
- Installing symfony/service-contracts (v3.5.1): Extracting archive
- Installing symfony/console (v7.2.1): Extracting archive
- Installing nunomaduro/termwind (v2.3.0): Extracting archive
- Installing voku/portable-ascii (2.0.3): Extracting archive
- Installing phar-io/phpoption (1.9.3): Extracting archive
- Installing graham-campbell/result-type (v1.1.3): Extracting archive
- Installing vlucas/phpdotenv (v5.6.1): Extracting archive
- Installing symfony/css-selector (v7.2.0): Extracting archive
- Installing tijsverkoyen/css-to-inline-styles (v2.3.0): Extracting archive
- Installing symfony/var-dumper (v7.2.3): Extracting archive
- Installing symfony/polyfill-uuid (v1.31.0): Extracting archive
- Installing symfony/uid (v7.2.0): Extracting archive
- Installing symfony/routing (v7.2.3): Extracting archive
```

Figure 2.1 Successful creation of Laravel project

4. Navigate to `C:/xampp/htdocs/ordersystem` in the command prompt.
5. Launch NetBeans and open the **ordersystem** project that you created.

Note: To use this framework, you need a text editor. As a text editor plays a key role in the entire process of coding, it is important to know some of the best code editors for Laravel.

<https://designwebkit.com/web-and-trends/5-best-code-editors-for-laravel/>

Optional: step 5-9 – To create an authentication system in Laravel using Breeze, follow this step-by-step guide. Laravel Breeze provides simple and minimal authentication scaffolding that is easy to set up and customize.

5. Install Laravel Breeze

```
composer require laravel/breeze --dev
```

6. After Breeze is installed, you need to run the Breeze install command to generate the authentication views and controllers:

```
php artisan breeze:install
```

7. Breeze uses frontend scaffolding that requires NPM. To install the required packages, run the following:

```
npm install
```

8. Now, run the command below for asset compilation

```
/* for development */
```

```
npm run dev
```

```
/* for production */
```

```
npm run production
```

or step 7 – 8 combined `npm install && npm run dev`

9. Breeze comes with the default authentication tables (like users, password resets, etc.) defined in Laravel migrations. To create these tables in your database, run:

```
php artisan migrate
```

C. Create a Local MySQL Database with the Appropriate Credentials

Using XAMPP or MySQL Workbench, create a database named **ordersystem** with username **root** and password **secret**.

D. Set Up the MySQL Database in the .env File

1. In Laravel 12, there is a default database connection using SQLite, but if we want to use MySQL instead, we need to add a MySQL connection with the database name, username, and password to the '.env' file.
2. Ensure that the .env file has the correct database, username, and password values:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ordersystem
DB_USERNAME=root
DB_PASSWORD=
```

The location of the .env file is shown in Figure 2.2.

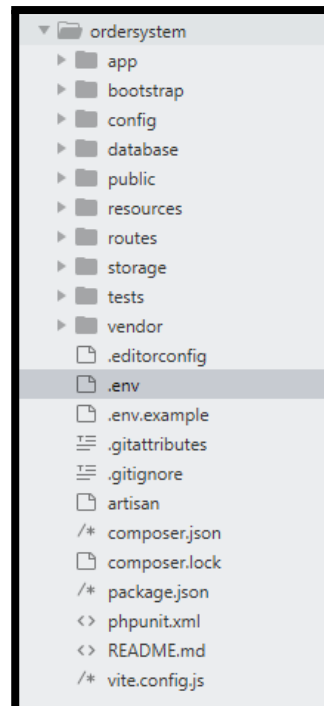


Figure 2.2 Location of .env file

3. At your terminal, change directory to the Laravel project directory, i.e. **ordersystem**.

4. Enter the following command:

php artisan migrate

Note: php artisan migrate **publishes all our schema to the database**. This command also creates the tables in the database.

The above statement will build 2 tables in your database

- **users**
- **password_resets**

and you should see the following output shown in Figure 2.4 below:

```
C:\xampp\htdocs\ordersystem>php artisan migrate

  INFO  Preparing database.

Creating migration table .....

  INFO  Running migrations.

0001_01_01_000000_create_users_table .....
0001_01_01_000001_create_cache_table .....
0001_01_01_000002_create_jobs_table .....
```

Figure 2.5 Running migrations

E. Construct a Model, Migration File and Controller for the Products Table

Note: Laravel is used by many thousands of developers every day to create all kinds of websites and applications. But fortunately, there are some very commonly used naming conventions that are followed by many developers when naming their Laravel project's variables, methods, and functions.

Naming database tables in Laravel

- DB tables should be in lowercase, with underscores to separate words (snake_case), and should be in plural form.

👍 For example: `posts` , `project_tasks` , `uploaded_images` .

❗ Bad examples: `all_posts` , `Posts` , `post` , `blogPosts`

Naming Models in Laravel

- A model should be in PascalCase/CapitalCase. They should be singular, no spacing between words, and capitalised.

👍 For example: `User` (`\App\User` OR `\App\Models\User` , etc), `ForumThread` , `Comment` .

❗ Bad examples: `Users` , `ForumPosts` , `blogpost` , `blog_post` , `Blog_posts` .

Naming Controllers

- Controllers should be in PascalCase/CapitalCase.
- They should be in singular case, with no spacing between words, and end with "Controller".
- Also, each word should be capitalised (i.e. `BlogController`, not `blogcontroller`).

👍 For example: `BlogController` , `AuthController` , `UserController` .

❗ Bad examples: `UsersController` (because it is in plural), `Users` (because it is missing the Controller suffix).

Ref: <https://webdevetc.com/blog/laravel-naming-conventions/>

1. Type the following command in your terminal:

```
php artisan make:model Product -m
```

The above command will create 2 files:

- **Product.php**— the PHP class representing the model
- **create_products_table** migration file

and you should see the output shown in Figure 2.6:

Note: A migration is the database table schema, it simply describes the database columns.

Note: `-m` or `— migration` □ Create a new migration file for the model.

Note: Model is used to get the data from the database.

```
C:\xampp\htdocs\ordersystem>php artisan make:model Product -m
INFO Model [C:\xampp\htdocs\ordersystem\app\Models\Product.php] created successfully.
INFO Migration [C:\xampp\htdocs\ordersystem\database\Migrations\2025_03_04_081718_create_products_table.php] created successfully.
```

Figure 2.6: Constructing a model

2. Create the schema for the products table.

- a. Navigate to **Laravel** >> **database** >> **migrations** >> **create_products_table**(refer to Figure 2.7).

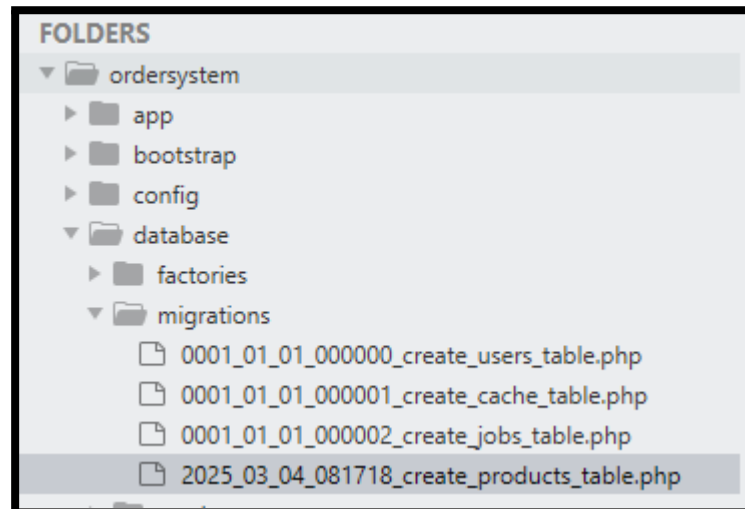


Figure 2.7 Locating create_products_table.php

- b. Complete the function **up()** as shown in Figure 2.8 below and save your file.

Note: A migration class contains two methods: **up** and **down**. The up method is used to add new tables, columns, or indexes to your database, while the down method should reverse the operations performed by the up method.

```
public function up()
{
    Schema::create('products', function (Blueprint $table) {
        $table->bigIncrements('id');
        $table->integer('code')->unique();
        $table->string('name');
        $table->timestamps();
    });
}
```

Figure 2.8 Code for function up()

Note: Laravel automatically maps a model to a database table using the pluralized form of the model's name.

- c. Migrate the table by entering the following command at the terminal:

php artisan migrate

and you should see the following output:

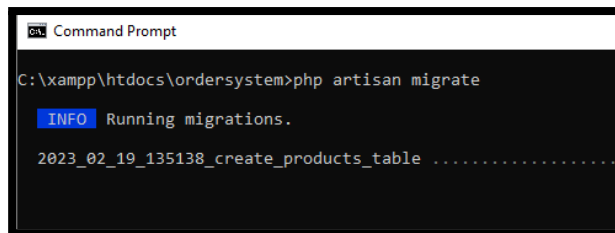


Figure 2.9: Migrating the table at the command prompt

Note: *php artisan migrate* publishes all our schema to the database. This command also creates the table in the database.

In the database, you should see the **products** table as shown in Figure 2.10 below.

Table	Action	Rows	Type	Collation	Size	Over
☐ cache	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	
☐ cache_locks	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	
☐ failed_jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	
☐ jobs	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	
☐ job_batches	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	
☐ migrations	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	16.0 KiB	
☐ password_reset_tokens	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	
☐ sessions	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	
☐ users	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	

Figure 2.10: List of tables in the ordersystem database

- Go to product model and type the following code
`protected $fillable = ['code', 'name'];`

F. Build View File to Add Information in the Database

- In the **resources >> views** folder(see Figure 2.11),

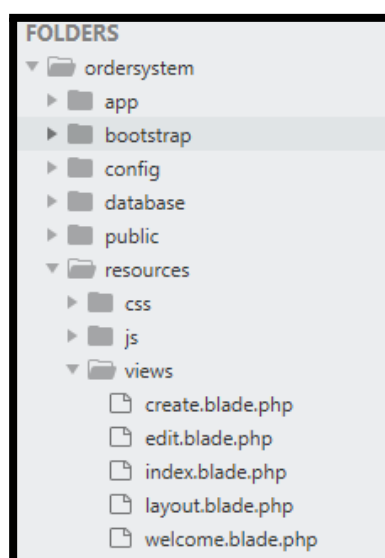


Figure 2.11 Location of the **views** folder

Create a PHP webpage named **layout.blade.php** with the code shown in Figure 2.12:

```

1 <!DOCTYPE html>
2 <html>
3 <head>
4     <title>Sample CRUD Application</title>
5     <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/
6     bootstrap.min.css" rel="stylesheet">
7     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome/6.5.1/css/all.min.css" />
8 </head>
9 <body>
10 <div class="container">
11     @yield('content')
12 </div>
13
14 </body>
15 </html>

```

Figure 2.12 Code for **layout.blade.php**

Create a PHP webpage named **create.blade.php** with the code shown in Figure 2.13:

Note: Blade is the simple yet powerful templating engine provided by Laravel. Blade view files use the *.blade.php* file extension and are typically stored in the *resources/views* directory.

```

1 <!-- create.blade.php -->
2
3 @extends('layout')
4 @section('content')
5
6
7     <h2>Add New Product</h2><br/>
8     <form method="post" action="{{ route('products.store') }}">
9         @csrf
10        <p>
11            <label for="code">Product Code:</label>
12            <input type="text" name="code">
13        </p>
14        <p>
15            <label for="name">Product Name:</label>
16            <input type="text" name="name">
17        </p>
18        <p>
19            <button type="submit">Submit</button>
20        </p>
21    </form>

```

Figure 2.13 Code for **create.blade.php**

```
<form method="post" action="{{ route('products.store') }}">
```

- `action="{{ route('products.store') }}"`
 - This dynamically generates the form's action URL using Laravel's `route()` helper.
 - The `route()` helper function in Laravel is used to generate URLs for named routes. Instead of hardcoding URLs in your Blade templates, you can use `route('route_name')` to ensure your application remains flexible and maintainable.
- `@csrf`
 - Adds a CSRF (Cross-Site Request Forgery) token for security in Laravel.
 - This prevents malicious requests from being submitted.

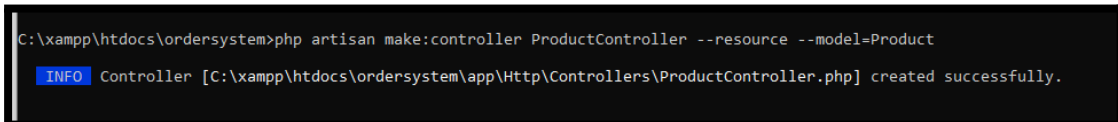
G. Create Controller and Route for Adding Products

1. Type the following command in your terminal:

```
php artisan make:controller ProductController --resource --model=Product
```

The `artisan make:controller` command in Laravel is used to generate a controller. When you add the `--resource` and `--model=Product` flags, Laravel will create a resource controller with methods for handling CRUD (Create, Read, Update, Delete) operations, and it will also bind the controller to the specified model.

You should see the message shown in Figure 2.14 if successfully executed:



```
C:\xampp\htdocs\ordersystem>php artisan make:controller ProductController --resource --model=Product
INFO Controller [C:\xampp\htdocs\ordersystem\app\Http\Controllers\ProductController.php] created successfully.
```

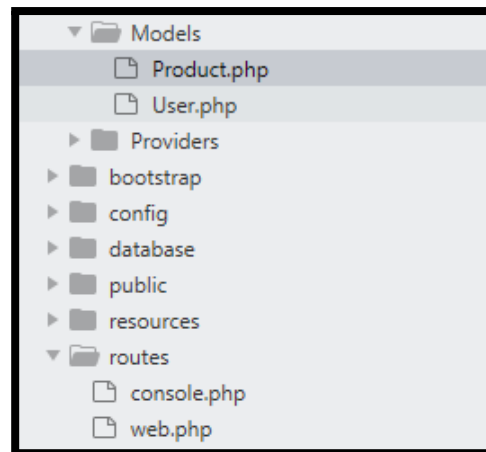
Figure 2.14: Screenshot of terminal after successful creation of the controller file **ProductController.php**

You will find a new file in this path "*app/Http/Controllers/ProductController.php*".

2. In this controller, seven methods will be created by default as follows:

- `index()` - Display a listing of the resource.
- `create()` - Show the form for creating a new resource.
- `store()` - Store a newly created resource in storage.
- `show()` - Display the specified resource.
- `edit()` - Show the form for editing the specified resource.
- `update()` - Update the specified resource in storage.
- `destroy()` - Remove the specified resource from storage.

3. To register the route for **ProductController**, navigate to **routes >> web.php**(see Figure 2.15).

Figure 2.15: Location of the **web.php** file

and add the following statement:

```
Route::resource('products', App\Http\Controllers\ProductController::class);
```

Note: The **routes/web.php** file defines routes that are for your web interface.

More Info: <https://www.cloudways.com/blog/routing-in-laravel/>

Your **routes/web.php** file automatically generates RESTful routes for the **ProductController**, including:

HTTP Method	URL	Action	Route Name
GET	/products	index	products.index
GET	/products/create	create	products.create
POST	/products	store	products.store
GET	/products/{product}	show	products.show
GET	/products/{product}/edit	edit	products.edit
PUT/PATCH	/products/{product}	update	products.update
DELETE	/products/{product}	destroy	products.destroy

4. At your terminal, type the following command:

php artisan route:list

You should see the following output:

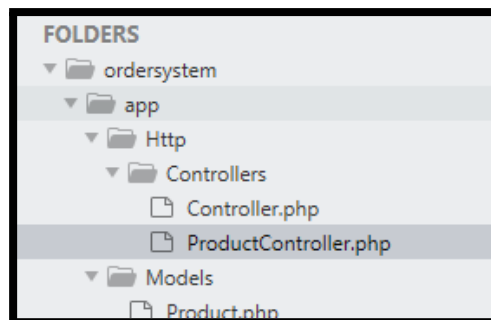
```

C:\xampp\htdocs\ordersystem>php artisan route:list
GET|HEAD / ..... products.index > ProductController@index
GET|HEAD products ..... products.store > ProductController@store
POST products/create ..... products.create > ProductController@create
GET|HEAD products/{product} ..... products.show > ProductController@show
GET|HEAD products/{product} ..... products.update > ProductController@update
PUT|PATCH products/{product} ..... products.destroy > ProductController@destroy
DELETE products/{product}/edit ..... products.edit > ProductController@edit
GET|HEAD storage/{path} ..... storage.local
GET|HEAD up .....

```

Figure 2.16: display of the method, URI, name, action and middleware for the routes

5. Go to the **ProductController.php** file



a. Add the following code in the **create()** function:

```

public function create() {
    return view('create');
}

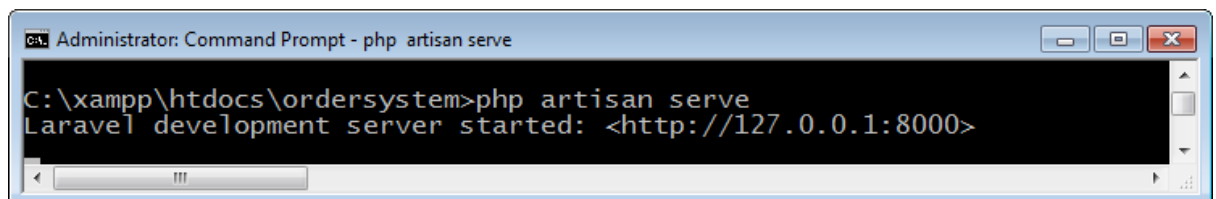
```

6. Next, start up the Laravel Development server by typing the following command at the terminal:

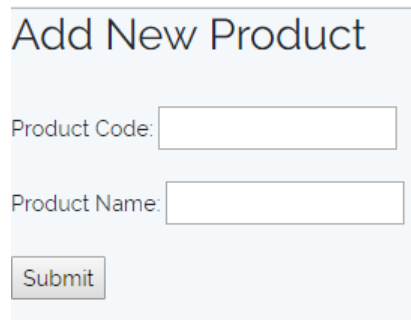
php artisan serve

Note: The Laravel PHP artisan serve command helps run applications on the PHP development server

You should see the following output:



7. Then, go to the browser and enter this URL: <http://localhost:8000/products/create>
You should be able to see the following webpage:



A web form titled "Add New Product" with a light blue background. It contains two input fields: "Product Code:" and "Product Name:". Below these fields is a "Submit" button.

H. Save Data into the Database

1. In `ProductController.php`, complete the function `store()` as follows:

```
public function store(Request $request) {  
    Product::create($request->only(['name', 'code']));  
    return redirect()->route('products.index')->with('success', 'Product  
        added successfully');  
}
```

2. Then, go to the browser and enter this URL: <http://localhost:8000/products/create> and enter a few records.

I. Create an Index Page that Lists all Products

1. In `ProductController.php`, add the following code to the function `index()` to retrieve data and return it to the index view:

```
public function index() {  
    $products = Product::all();  
    return view('index', compact('products'));  
}
```

2. In resources >> views, create a php webpage named `index.blade.php` with the following code:

```

1 <!-- index.blade.php -->
2 @extends('layout')
3
4 @section('content')
5 <div class="container">
6     <br />
7     @if (\Session::has('success'))
8     <div class="alert alert-success">
9         <p>{{ \Session::get('success') }}</p>
10    </div><br />
11    @endif
12
13    <table class="table table-striped">
14        <thead>
15            <tr>
16                <th>ID</th>
17                <th>Product Code</th>
18                <th>Product Name</th>
19                <th colspan="2">Action</th>
20            </tr>
21        </thead>
22        <tbody>
23            @foreach($products as $product)
24            <tr>
25                <td>{{ $product->id }}</td>
26                <td>{{ $product->code }}</td>
27                <td>{{ $product->name }}</td>

```

Note: The remaining code is found in the next page

```

29            <td>
30                <a href="{{ route('products.edit', $product->id) }}" class
31                    ="btn btn-warning">Edit</a>
32            </td>
33            <td>
34                <form action="{{ route('products.destroy', $product->id)
35                    }}" method="post">
36                    @csrf
37                    @method('DELETE')
38                    <button class="btn btn-danger" type="submit">Delete</
39                    button>
40                </form>
41            </td>
42        </tr>
43    @endforeach
44 </tbody>
45 </table>
46 </div>

```

If go to the browser and type in the URL <http://localhost:8000/products/create>, you should be able to see the list of products that exist in your **products** table in the database.

ID	Product Code	Product Name	Action	
1	1001	Pink Rose Vase	<button>Edit</button>	<button>Delete</button>
2	1002	Ceramic Bowl	<button>Edit</button>	<button>Delete</button>

J. Build an Edit View for Updating Product Information

1. Go to the **ProductController.php** file. In the function **edit()**, complete the code as follows:

```
public function edit($id) {
    $product = Product::find($id);
    return view('edit', compact('product', 'id'));
}
```

2. Create an **edit.blade.php** file inside **resources >>views** (see below):

```
1 <!-- edit.blade.php -->
2
3 @extends('layout')
4 @section('content')
5
6     <h2>Edit Product Details</h2><br />
7     <form method="post" action="{{ route('products.update', $id) }}">
8         @csrf
9         <input name="_method" type="hidden" value="PATCH">
10        <p>
11            <label for="code">Product Code: </label>
12            <input type="text" name="code" value="{{ $product->code }}">
13        </p>
14
15        <p>
16            <label for="name">Product Name: </label>
17            <input type="text" name="name" value="{{ $product->name }}">
18        </p>
19        <p>
20            <button type="submit" style="margin-left: 38px">Update</button>
21        </p>
22    </form>
23
24
```

3. Code the function `update()` in `ProductController.php`.

```
public function update(Request $request, $id) {  
    $product = Product::findOrFail($id);  
    $product->update($request->only(['name', 'code']));  
    return redirect()->route('products.index');  
}
```

K. Complete the Delete Product Function

In `ProductController.php`, complete the function `destroy()` as follows:

```
public function destroy($id) {  
    Product::findOrFail($id)->delete();  
    return redirect()->route('products.index')->with('success', 'Product  
        deleted successfully');  
}
```